

15. ETAPA 10 — Implementación de actividad

Requisitos de instalación y software necesario

Para poder ejecutar el sistema en otra computadora es necesario tener instalados algunos programas y herramientas. Los que se utilizaron durante el desarrollo del proyecto son:

- Node.js, versión 22.22.3
- pnpm, utilizado principalmente para instalar y manejar las dependencias del Backend.
- npm, que viene incluido con Node.js y se utiliza para el Frontend.
- PostgreSQL, que es el sistema de base de datos utilizado por el proyecto.
- Angular CLI, que se puede instalar con el comando `npm install -g @angular/cli`.
- Git, para poder clonar el repositorio desde GitHub.
- Visual Studio Code, utilizado como editor de código durante el desarrollo.



Dependencias del proyecto

El sistema está dividido en dos partes principales: **Backend** y **Frontend**, por lo que cada uno tiene sus propias dependencias y se ejecuta por separado.

Backend

El Backend fue desarrollado utilizando **Node.js**, **Express** y **TypeScript**. Para conectarse a PostgreSQL se utiliza la librería `pg` y también se utiliza `dotenv` para manejar las variables de entorno.

Para instalar las dependencias relacionadas con PostgreSQL se utilizaron los siguientes comandos:

```
pnpm install
```

```
pnpm add pg dotenv
```

```
pnpm add -D @types/pg @types/node
```

Para instalar **Express**, que se utiliza para crear y manejar el servidor del Backend:

```
pnpm add express
```

```
pnpm add -D @types/express
```

También se instaló **CORS**, que permite que el Frontend pueda realizar peticiones al Backend desde otro origen:

```
pnpm add cors
```

```
pnpm add -D @types/cors
```

Además de estas librerías, el Backend utiliza otras dependencias para diferentes funciones del sistema. Por ejemplo, jsonwebtoken se utiliza para manejar los tokens de autenticación y bcryptjs para proteger las contraseñas de los usuarios.

Frontend

El Frontend fue desarrollado con **Angular** y utiliza diferentes librerías para facilitar el desarrollo de la interfaz.

Para instalar **Angular Material**, que se utiliza para componentes como botones, formularios, ventanas, iconos y otros elementos de la interfaz, se ejecuta:

```
ng add @angular/material
```

También se utiliza **Bootstrap** para algunos estilos y elementos visuales. Para instalarlo se utiliza:

```
npm install bootstrap
```

Una vez instaladas estas dependencias, el Frontend puede comunicarse con el Backend mediante peticiones HTTP para consultar, guardar, modificar y eliminar información.

Configuración de la base de datos

Antes de ejecutar el sistema es necesario tener creada la base de datos en PostgreSQL, ya que el Backend necesita conectarse a ella para funcionar correctamente.

Para configurar la base de datos se realizaron los siguientes pasos:

1. Crear una base de datos nueva en PostgreSQL.
2. Ejecutar los archivos .sql incluidos en el proyecto para crear las tablas y los procedimientos necesarios.

3. Comprobar que la conexión entre el Backend y la base de datos funcione correctamente.

En este proyecto se utilizó una base de datos llamada siged, aunque el nombre puede cambiar dependiendo de la configuración que se utilice en cada computadora.

Query	Query History	Query	Query History
1 -- Eliminar tablas existentes		31	
2		32 -- tabla usuario	
3 drop table if exists historial_incidente cascade;		33 create table usuario (	
4 drop table if exists alerta cascade;		34 id bigint generated by default as identity primary key,	
5 drop table if exists asignacion_personal cascade;		35 nombre varchar(120) not null,	
6 drop table if exists asignacion_recurso cascade;		36 email varchar(160) not null unique,	
7 drop table if exists recurso cascade;		37 password_hash varchar(255) not null,	
8 drop table if exists refugio cascade;		38 telefono varchar(30),	
9 drop table if exists incidente cascade;		39 rol varchar(30) not null default 'VOLUNTARIO'	
10 drop table if exists zona cascade;		40 check (rol in ('ADMIN', 'COORDINADOR', 'RESCATISTA', 'VOLUNTARIO', 'GESTOR_REFUGIO')),	
11 drop table if exists usuario cascade;		41 activo boolean not null default true,	
12		42 habilidades text,	
13 drop function if exists sp_listar_usuarios() cascade;		43 disponible boolean not null default false,	
14 drop function if exists sp_buscar_usuario(bigint) cascade;		44 created_at timestamptz not null default now(),	
15 drop function if exists sp_listar_zonas() cascade;		45 updated_at timestamptz	
16 drop function if exists sp_buscar_zona(bigint) cascade;		46);	
17 drop function if exists sp_listar_incidentes() cascade;		47	
18 drop function if exists sp_buscar_incidente(bigint) cascade;		48 -- tabla zona	
19 drop function if exists sp_listar_refugios() cascade;		49 create table zona (	
20 drop function if exists sp_buscar_refugio(bigint) cascade;		50 id bigint generated by default as identity primary key,	
21 drop function if exists sp_listar_recursos() cascade;		51 nombre varchar(120) not null,	
22 drop function if exists sp_buscar_recurso(bigint) cascade;		52 municipio varchar(120),	
23 drop function if exists sp_listar_asignaciones_recurso() cascade;		53 departamento varchar(120),	
24 drop function if exists sp_buscar_asignacion_recurso(bigint) cascade;		54 pais varchar(80) not null default 'Guatemala',	
25 drop function if exists sp_listar_asignaciones_personal() cascade;		55 latitud numeric(9,6),	
		56 longitud numeric(9,6)	

Variables de entorno

Dentro de la carpeta del Backend se debe crear un archivo llamado .env. En este archivo se colocan los datos necesarios para configurar el servidor y la conexión con PostgreSQL.

Las variables utilizadas son:

- PORT: puerto donde se ejecuta el Backend. En este caso se utiliza el puerto 3000.
- NODE_ENV: indica el entorno en el que se está ejecutando el proyecto. Se utiliza development.
- DB_HOST: dirección del servidor de PostgreSQL, normalmente localhost.
- DB_PORT: puerto utilizado por PostgreSQL, normalmente 5432.
- DB_NAME: nombre de la base de datos, en este caso siged.
- DB_USER: usuario de PostgreSQL, normalmente postgres.
- DB_PASSWORD: contraseña del usuario de PostgreSQL.
- DB_POOL_MIN: cantidad mínima de conexiones del pool. Se utiliza 2.
- DB_POOL_MAX: cantidad máxima de conexiones del pool. Se utiliza 10.
- JWT_SECRET: clave utilizada para generar y validar los tokens de inicio de sesión.
- JWT_EXPIRES_IN: tiempo de duración del token. En este caso es 24h.

- CORS_ORIGIN: dirección desde donde se ejecuta el Frontend. Se utiliza `http://localhost:4200`.

Los valores pueden cambiar dependiendo de la computadora donde se instale el proyecto. Principalmente, la contraseña de PostgreSQL y el JWT_SECRET deben ser configurados por cada usuario y no deben compartirse.

```
Backend > .env
1  PORT=3000
2
3  NODE_ENV=development
4
5  DB_HOST=localhost
6
7  DB_PORT=5432
8
9  DB_NAME=siged
10
11 DB_USER=postgres
12
13 DB_PASSWORD=admin
14
15 DB_POOL_MIN=2
16
17 DB_POOL_MAX=10
18
19 JWT_SECRET=clave_secreta_para_jwt
20
21 JWT_EXPIRES_IN=24h
22
23 CORS_ORIGIN=http://localhost:4200
24
25
```

Usuarios iniciales

El proyecto incluye un script para crear algunos usuarios de prueba. Esto permite probar los diferentes roles del sistema sin tener que registrarlos manualmente.

El script se ejecuta desde la carpeta del Backend con el siguiente comando: `pnpm exec tsx src/seed.ts`

Al ejecutarlo se crean usuarios correspondientes a los diferentes roles que tiene el sistema, entre ellos:

- Administrador
- Rescatista
- Gestor de Refugio
- Voluntario

De esta manera se puede ingresar al sistema con diferentes roles y comprobar las funciones disponibles para cada uno.

```
-- Usuarios
call sp_agregar_usuario('carlos mendez', 'carlos.mendez@sighed.com', '$2b$10$hash123admin', '5555-1001', 'ADMIN', 'ge
call sp_agregar_usuario('maría gonzález', 'maria.gonzalez@sighed.com', '$2b$10$hash123coord', '5555-1002', 'COORDINAC
call sp_agregar_usuario('juan perez', 'juan.perez@sighed.com', '$2b$10$hash123rescate', '5555-1003', 'RESCATISTA', 'r
call sp_agregar_usuario('ana rodríguez', 'ana.rodriguez@sighed.com', '$2b$10$hash123gestor', '5555-1004', 'GESTOR_REF
call sp_agregar_usuario('luis martínez', 'luis.martinez@sighed.com', '$2b$10$hash123voluntario', '5555-1005', 'VOLUNT

com', '$2b$10$hash123admin', '5555-1001', 'ADMIN', 'gestión de emergencias, coordinación', true);
d.com', '$2b$10$hash123coord', '5555-1002', 'COORDINADOR', 'coordinación de equipos, logística', true);
'$2b$10$hash123rescate', '5555-1003', 'RESCATISTA', 'rescate acuático, primeros auxilios', true);
com', '$2b$10$hash123gestor', '5555-1004', 'GESTOR_REFUGIO', 'gestión de albergues, atención a damnificados', true);
com', '$2b$10$hash123voluntario', '5555-1005', 'VOLUNTARIO', 'apoyo general, distribución de ayuda', true);
```

Procedimiento de instalación

Para instalar y ejecutar el proyecto se siguieron estos pasos:

1. Clonar el repositorio del proyecto desde GitHub.
2. Entrar a la carpeta del Backend.
3. Instalar las dependencias ejecutando `pnpm install`.
4. Crear el archivo `.env` con los datos de configuración.
5. Ejecutar el script para crear los usuarios de prueba.
6. Levantar el Backend con `pnpm run dev`.
7. Entrar a la carpeta del Frontend.
8. Instalar las dependencias con `npm install`.
9. Levantar el Frontend utilizando `ng serve`.
10. Abrir el navegador en `http://localhost:4200`.

11. Iniciar sesión con alguno de los usuarios de prueba y comprobar que las diferentes funciones del sistema trabajen correctamente.

Con estos pasos se puede dejar preparado el proyecto para realizar las pruebas del sistema en otra computadora.

```
C:\cmdr_mini\cmdr_mini (master) (cmdr_mini@1.0.0)
λ cd C:\Users\wilfr\OneDrive\Desktop

C:\Users\wilfr\OneDrive\Desktop (master)
λ git clone https://github.com/KevinLancerio-2022063/Proyecto_Integrador_BIM_4.git
Cloning into 'Proyecto_Integrador_BIM_4'...
remote: Enumerating objects: 29043, done.
remote: Counting objects: 100% (1165/1165), done.
remote: Compressing objects: 100% (530/530), done.
Remote: Total 29043 (delta 780), reused 940 (delta 625), pack-reused 27878 (from 2)
Receiving objects: 100% (29043/29043), 84.13 MiB | 5.87 MiB/s, done.
Resolving deltas: 100% (9614/9614), done.

C:\Users\wilfr\OneDrive\Desktop (master)
λ cd C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4

C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4 (main -> origin)
λ git checkout develop
Updating files: 100% (470/470), done.
branch 'develop' set up to track 'origin/develop'.
Switched to a new branch 'develop'
```

```
PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Backend> pnpm install
Already up to date
```

```
Update available! 11.21.0 → 12.5.1.
Changelog: https://pnpm.io/v/12.5.1
To update, run: pnpm self-update
```

```
Done in 1.7s using pnpm v11.21.0
```

```
PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Backend> █
```

```
Backend > .env
1  PORT=3000
2
3  NODE_ENV=development
4
5  DB_HOST=localhost
6
7  DB_PORT=5432
8
9  DB_NAME=siged
10
11 DB_USER=postgres
12
13 DB_PASSWORD=admin
14
15 DB_POOL_MIN=2
16
17 DB_POOL_MAX=10
18
19 JWT_SECRET=clave_secreta_para_jwt
20
21 JWT_EXPIRES_IN=24h
22
23 CORS_ORIGIN=http://localhost:4200
24
25
```

```

Done in 1.7s using pnpm v1.21.0
● PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Backend> pnpm tsx src/seed.ts
  ◇ injected env (12) from .env // tip: % enable debugging { debug: true }
  Iniciando seed de usuarios...
  Nueva conexión establecida con PostgreSQL
  ✓ Usuario creado: carlos.mendez@sighed.com | password: admin123
  ✓ Usuario creado: maria.gonzalez@sighed.com | password: coord123
  ✓ Usuario creado: juan.perez@sighed.com | password: rescatista123
  ✓ Usuario creado: ana.rodriguez@sighed.com | password: gestor123
  ✓ Usuario creado: luis.martinez@sighed.com | password: voluntario123

  Seed completado. Credenciales:
    admin@sighed.com      → admin123
    maria.gonzalez@sighed.com → coord123
    juan.perez@sighed.com  → rescatista123
    ana.rodriguez@sighed.com → gestor123
    luis.martinez@sighed.com → voluntario123
○ PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Backend>

```

```

○ PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Backend> pnpm run dev
$ tsx src/api/server.ts
  ◇ injected env (12) from .env // tip: % suppress logs { quiet: true }
  ◇ injected env (8) from .env // tip: % multiple files { path: ['.env.local', '.env'] }
  Iniciando SIGED Server...
  Nueva conexión establecida con PostgreSQL
  Conexión exitosa a PostgreSQL
  Fecha del servidor: 2026-09-22T03:58:43.272Z
  Conexion a PostgreSQL establecida
  =====
  Servidor corriendo en puerto 3000
  Entorno: development
  URL: http://localhost:3000
  API Health: http://localhost:3000/api/health
  Auth: http://localhost:3000/api/auth/login
  =====
  █

```

```

● PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Frontend> npm install

up to date, audited 394 packages in 15s

112 packages are looking for funding
  run `npm fund` for details

3 vulnerabilities (2 moderate, 1 high)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
🔧 PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Frontend>

```

```
PS C:\Users\wilfr\OneDrive\Desktop\Proyecto_Integrador_BIM_4\Frontend> ng serve --open
```

Browser bundles

Initial chunk files	Names	Raw size
main.js	main	29.34 kB
styles.css	styles	9.70 kB
chunk-KDGBFBQC.js	-	4.22 kB
chunk-CZJLB7T5.js	-	1.13 kB
chunk-XFQ52ZJ6.js	-	236 bytes
Initial total		44.62 kB

Lazy chunk files	Names	Raw size
chunk-BHDHTION.js	logistica-module	501.13 kB
chunk-YHFJC6SD.js	operaciones-module	304.94 kB
chunk-WTJWDSBY.js	incidentes-module	233.95 kB
chunk-SHZD7QHB.js	zona-form-component	76.55 kB
chunk-GHHAYENF.js	zona-detail-component	65.62 kB
chunk-BVWPEJ24.js	zona-list-component	62.63 kB
chunk-IOIXEVGH.js	login-component	62.16 kB
chunk-JOHH5PTJ.js	usuario-list-component	58.33 kB
chunk-PWK4DNKR.js	register-component	48.48 kB
chunk-OKNIFNFQ.js	usuario-form-component	32.87 kB
chunk-MHPURLZW.js	usuario-detail-component	30.97 kB
chunk-5EYAHCUB.js	usuario-profile-component	26.25 kB
chunk-PYB34ZCB.js	-	4.27 kB
chunk-ZPUTPQSS.js	-	2.47 kB
chunk-TJ2H5OZL.js	-	2.25 kB

...and 5 more lazy chunks files. Use "--verbose" to show all the files.

Server bundles

Initial chunk files	Names	Raw size
main.server.mjs	main.server	30.71 kB
chunk-HH5AKI4B.mjs	-	4.20 kB
server.mjs	server	1.86 kB
chunk-XZXRA2IB.mjs	-	1.11 kB
polyfills.server.mjs	polyfills.server	268 bytes
chunk-IIPRAKI5.mjs	-	222 bytes

Lazy chunk files	Names	Raw size
chunk-7GLXYNO3.mjs	logistica-module	501.12 kB
chunk-MMLZPFBR.mjs	operaciones-module	304.92 kB
chunk-M6N7ZEXX.mjs	incidentes-module	233.94 kB
chunk-C7ZGGJCJ.mjs	zona-form-component	76.54 kB
chunk-FETHGLKC.mjs	zona-detail-component	65.61 kB
chunk-2LJMEKZI.mjs	zona-list-component	62.62 kB
chunk-TQJQJEUM.mjs	login-component	62.15 kB
chunk-2P6D3G3C.mjs	usuario-list-component	58.32 kB
chunk-WCQXXIRN.mjs	register-component	48.47 kB
chunk-U75FBNRN.mjs	usuario-form-component	32.86 kB
chunk-064YAF62.mjs	usuario-detail-component	30.96 kB
chunk-LD4P6VG3.mjs	usuario-profile-component	26.25 kB
chunk-DXLM2567.mjs	-	4.26 kB
chunk-3WADLNOZ.mjs	-	2.45 kB

```
▲ [WARNING] NG8113: TipoRecursoPipe is not used within the template of RecursoListComponent [plugin angular-compiler]

src/app/features/logistica/components/recursos/recurso-list/recurso-list.component.ts:32:4:
  32 |     TipoRecursoPipe,
     |     ~~~~~

▲ [WARNING] NG8113: UnidadMedidaPipe is not used within the template of RecursoListComponent [plugin angular-compiler]

src/app/features/logistica/components/recursos/recurso-list/recurso-list.component.ts:33:4:
  33 |     UnidadMedidaPipe,
     |     ~~~~~

▲ [WARNING] NG8113: EstadoRefugioPipe is not used within the template of RefugioListComponent [plugin angular-compiler]

src/app/features/logistica/components/refugios/refugio-list/refugio-list.component.ts:35:4:
  35 |     EstadoRefugioPipe,
     |     ~~~~~

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
→ Local: http://localhost:4200/
→ press h + enter to show help
```

